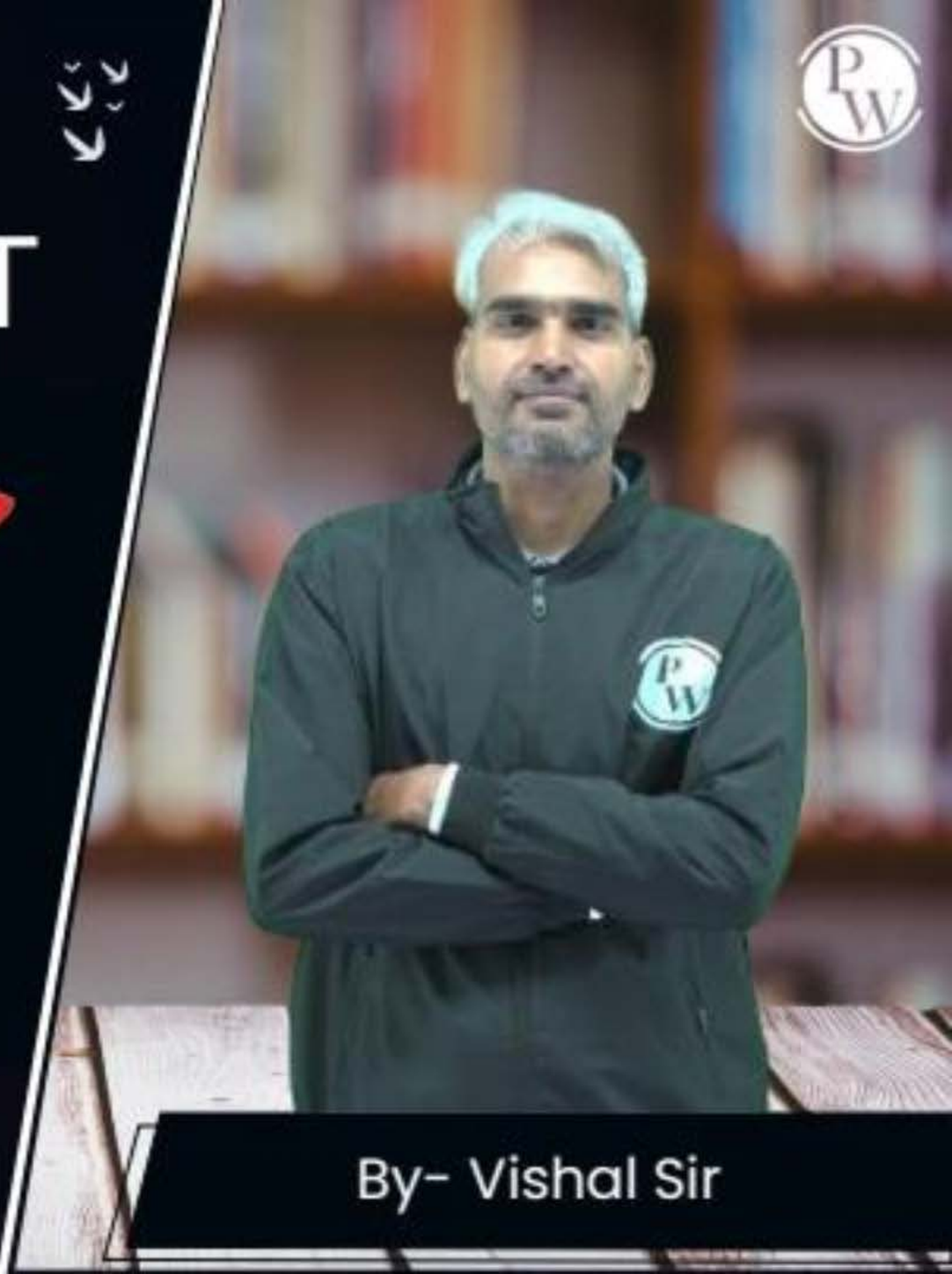
Computer Science & IT



Database Management System

Query Languages

Lecture No. 07



By- Vishal Sir

Recap of Previous Lecture



- Topic: SQL commands
- Topic: SQL clauses

Slide

Topics to be Covered



- Topic: SQL clauses
- Topic: Introduction to nested query
- Topic: IN, ANY, ALL and EXISTS operators

Slide



Topic : SQL clauses

1. **FROM** - The FROM clause in SQL is used to select the database tables

2. **WHERE** - The WHERE clause in SQL is used to select the tuples from the database table based on conditions specified with WHERE clause.
{ If "Where" Clause is not present, then all rows will be selected }

3. **GROUP BY** - GROUP BY clause is used to group the result of WHERE clause based on attributes specified with GROUP BY clause.
{ If Where Clause is not Present, then Group by Clause will be applied on all rows }
{ If Group by clause is not present, then we will assume that all tuples together form a single group }

4. **HAVING** - HAVING clause can be used with or without GROUP BY clause. If group by clause is present, then condition specified with having clause will be used to select the groups that satisfy the specified condition.
{ Having Condn is applied on each group }
{ If Having Clause is not present then all the groups will get selected }

5. **ORDER BY** - The ORDER BY clause in SQL is used for sorting the records of the database based on attributes specified with ORDER BY clause.



Topic : Order of execution

Order of Execution:-

1. From
2. Where
3. Group By
4. Having
5. Select
6. Order BY



Topic : SQL clauses

- FROM:- From clause is used to select the tables from the database.

Everything

```
Select *
From Student
```

⇓

O/p of this query will be Complete Student table

Student

Sid	Sname	Marks	Branch
S1	A	40	CS
S2	A	20	IT
S3	B	60	CS
S4	A	60	EC
S5	C	40	IT
S6	C	NULL	EC



Topic : SQL clauses

- FROM:- From clause is used to select the tables from the database.

- Select Sid
  From Student ⇒ O/P :

Sid
S1
S2
S3
S4
S5
S6

- Select distinct Sid
  From Student = O/P

Student

Sid	Sname	Marks	Branch
S1	A	40	CS
S2	A	20	IT
S3	B	60	CS
S4	A	60	EC
S5	C	40	IT
S6	C	NULL	EC



Topic : SQL clauses

- FROM:- From clause is used to select the tables from the database.

- Select Sname From Student ⇒ o/p :

Sname
A
A
B
A
C
C

- Select distinct Sname From Student ⇒ o/p :

Sname
A
B
C

Student

Sid	Sname	Marks	Branch
S1	A	40	CS
S2	A	20	IT
S3	B	60	CS
S4	A	60	EC
S5	C	40	IT
S6	C	NULL	EC



Topic : SQL clauses

WHERE:- Used to select the tuples from the database table based on conditions specified with WHERE clause.

* Retriere Sids of students who Scored More than 40 Marks

```
Select Sid
    From Student
        Where Marks > 40
```

⇒ O/P =

Sid
S3
S4

Student

Sid	Sname	Marks	Branch
S1	A	✗ 40	CS
S2	A	✗ 20	IT
S3	B	✓ 60	CS
S4	A	✓ 60	EC
S5	C	✗ 40	IT
S6	C	✗ NULL	EC



Topic : SQL clauses

WHERE:- Used to select the tuples from the database table based on conditions specified with WHERE clause.

* Retriere Sids of students where Marks are more than or equal to 20 And less than or equal to 40

```
Select Sid
From Student
Where (Marks >= 20 AND Marks <= 40)
```

Student

Sid	Sname	Marks	Branch
S1	A	40	CS
S2	A	20	IT
S3	B	60	CS
S4	A	60	EC
S5	C	40	IT
S6	C	NULL	EC



Topic : SQL clauses

GROUP BY:- GROUP BY clause is used to group the result of WHERE clause.

{If Where clause is not present then all the rows will be grouped based on attributes specified with group by}

Query: Retrieve names of all branches along with maximum marks in that branch.

Student

Sid	Sname	Marks	Branch
S1	A	40	CS
S2	A	20	IT
S3	B	60	CS
S4	A	60	EC
S5	C	40	IT
S6	C	NULL	EC

* Select Max (Marks)
  From Student

$\Rightarrow$ o/p =

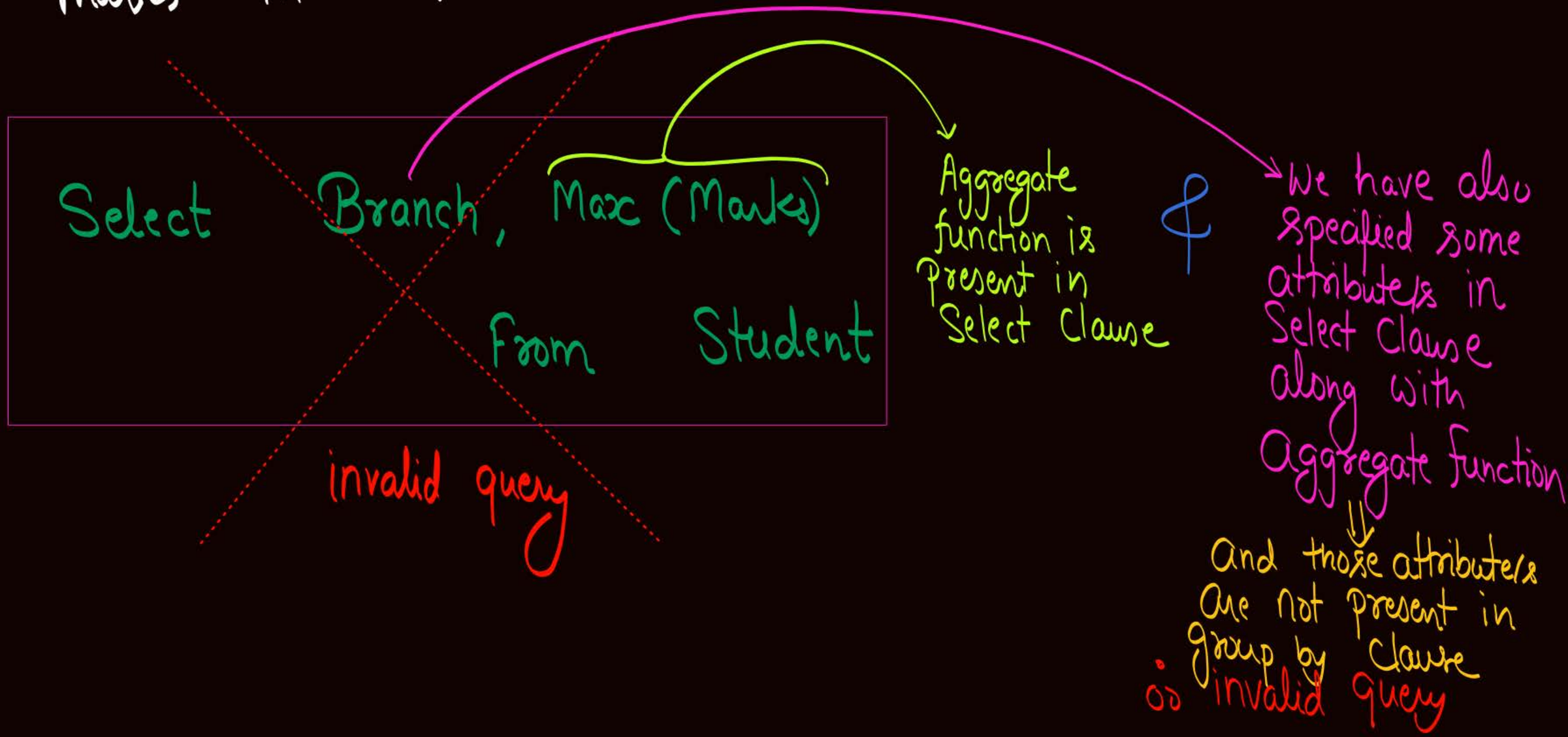
Max(Marks)
60

* Select distinct Branch
  From Student

$\Rightarrow$ o/p =

Branch
CS
IT
EC

→ Retrieve name of all Branches along with Maximum marks in that Branch.



Note:- If "group by" clause is present, then aggregate functions will be applied on each group.



Topic : SQL clauses

→ If attribute is present in Select Clause along with aggregate function, then that attribute must be present in group by clause.

NOTE:-

1. We can not select any attribute in SELECT clause along with aggregate function if those attributes are not present in GROUP BY clause.
   But, If attributes are present in Group by clause, then it is not necessary to select some or all of them using select clause along with aggregate function
2. If aggregate function is used along with GROUP BY clause, then aggregate function is applied on each group.



Topic : SQL clauses

GROUP BY:- GROUP BY clause is used to group the result of WHERE clause.

Query: Retrieve names of all branches along with maximum marks in that branch.

```
Select Branch, Max(Marks)
From Student
Group by (Branch)
```

Student

Sid	Sname	Marks	Branch
S1	A	40	CS
S2	A	20	IT
S3	B	60	CS
S4	A	60	EC
S5	C	40	IT
S6	C	NULL	EC

O/P :

Branch	Max(Marks)
CS	60
IT	40
EC	60



Topic : SQL clauses

- HAVING:- HAVING condition is applied on each group. (Provided Group by clause is present)

Query: Retrieve branch names with average marks more than or equal to 40.

Student

Sid	Sname	Marks	Branch
S1	A	40	CS
S2	A	20	IT
S3	B	60	CS
S4	A	60	EC
S5	C	40	IT
S6	C	NULL	EC

Avg(Marks) ✓ 50 ← CS ✓ ; IT ✗ ; CS ; × 30 ; EC ✓ 60 ← ; IT ; EC

```
Select Branch
From Student
Group by (Branch)
Having (Avg(Marks) ≥ 40)
```

⇒ O/P =

Branch
CS
EC



Topic : SQL clauses

ORDER BY:- This clause is used to sort the result in ascending or descending order based on values of attribute specified with ORDER BY clause.

By default order is ascending order.

Otherwise → For ascending order ⇒ ASC

Otherwise → For descending order ⇒ DESC

Student

Sid	Sname	Marks	Branch
S1	A	40	CS
S2	A	20	IT
S3	B	60	CS
S4	A	60	EC
S5	C	40	IT
S6	C	NULL	EC

```
Select Sid, Sname
From Student
Where (Marks ≥ 40)
```

⇒ o/p

Sid	Sname
S_1	A
S_3	B
S_4	A
S_5	C

```
Select Sid, Sname
From Student
Where (Marks ≥ 40)
Order by (Sname)
```

⇒ o/p :

By default Ascending order

Sid	Sname
S_1	A
S_4	A
S_3	B
S_5	C

```
Select Sid, Sname
From Student
Where (Marks ≥ 40)
Order by (Sname) DESC
```

⇒ o/p :

Sid	Sname
S_5	C
S_3	B
S_4	A
S_1	A

* Retrieve Sids of all the students who scored maximum Marks.

* Select Sid
  from Student
  Where (Marks = Max(Marks))

"Invalid query"

"Aggregate function" can not be used directly within the where Condition

Student

Sid	Sname	Marks	Branch
S1	A	40	CS
S2	A	20	IT
S3	B	60	CS
S4	A	60	EC
S5	C	40	IT
S6	C	NULL	EC

* Retriere Sids of all the students who scored maximum Marks.

```
Select   Sid
   From  Student
      Where (Marks = (Select  Max(Marks)
                        From  Student))
```



Topic : Nested queries

"Sub-query"

```
Select  Sid
From  Student
Where  ( Marks = (Select  Max (Marks)
                  From  Student) )
```

Outer query (Main query)

it will select the maximum marks from Complete Student table

inner query (Sub-query)

Nested Query

Independent Nested query (Independent Sub-query)

* If inner query can be executed independently, then it is called independent Sub-query

Correlated Nested query (Correlated Sub-query)

* When execution of inner query requires value of an attribute from the relation specified in Outer query. then it is called Correlated Sub-query

Nested Query

Independent Nested query (Independent Sub-query)

eg:
```
Select Sid
From Student
Where (Marks) = (Select Max(Marks)
                 From Student)
```

it can be executed independently
∴ Independent Sub-query

Correlated Nested query (Correlated Sub-query)

eg:
```
Select R.A
From R
Where ....( Select *
            From S
            Where (R.B = S.C))
```

'R' is the relation specified in outer query

Outer query

It can not be executed independently, because it requires an attribute from the relation specified in the outer query

⇒ Hence, Correlated Sub-query

(1) order of execution w.r.t. independent Sub-query

Inner query will be executed first
And it will produce its output,
then outer query will use the output
produced by inner query for its
execution.

✱ Retrieve Sids of all the students Who scored maximum Marks.

* Select Sid
  from Student
  Where (Marks = (Select Max(Marks) From Student))

inner query will execute first, and it will Produce 60 as o/p

60

↳ o/p =

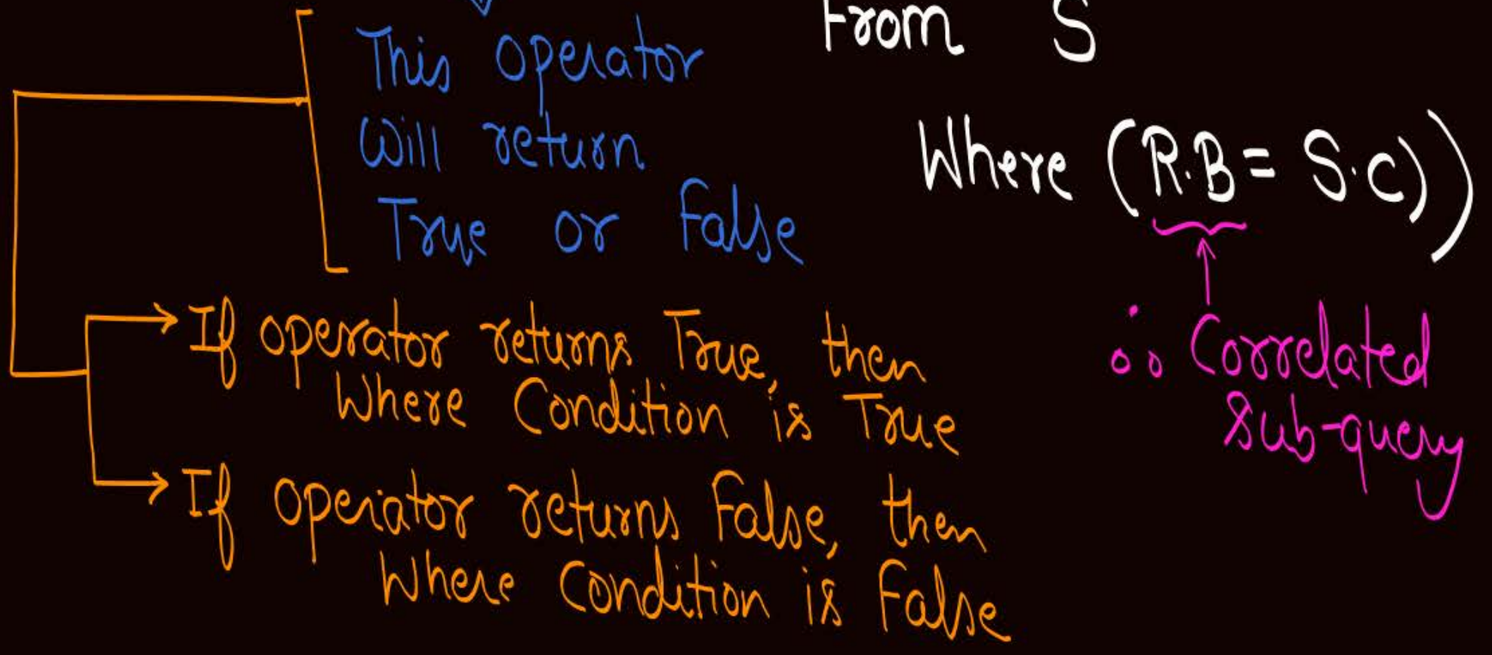
Sid
S3
S4

Student

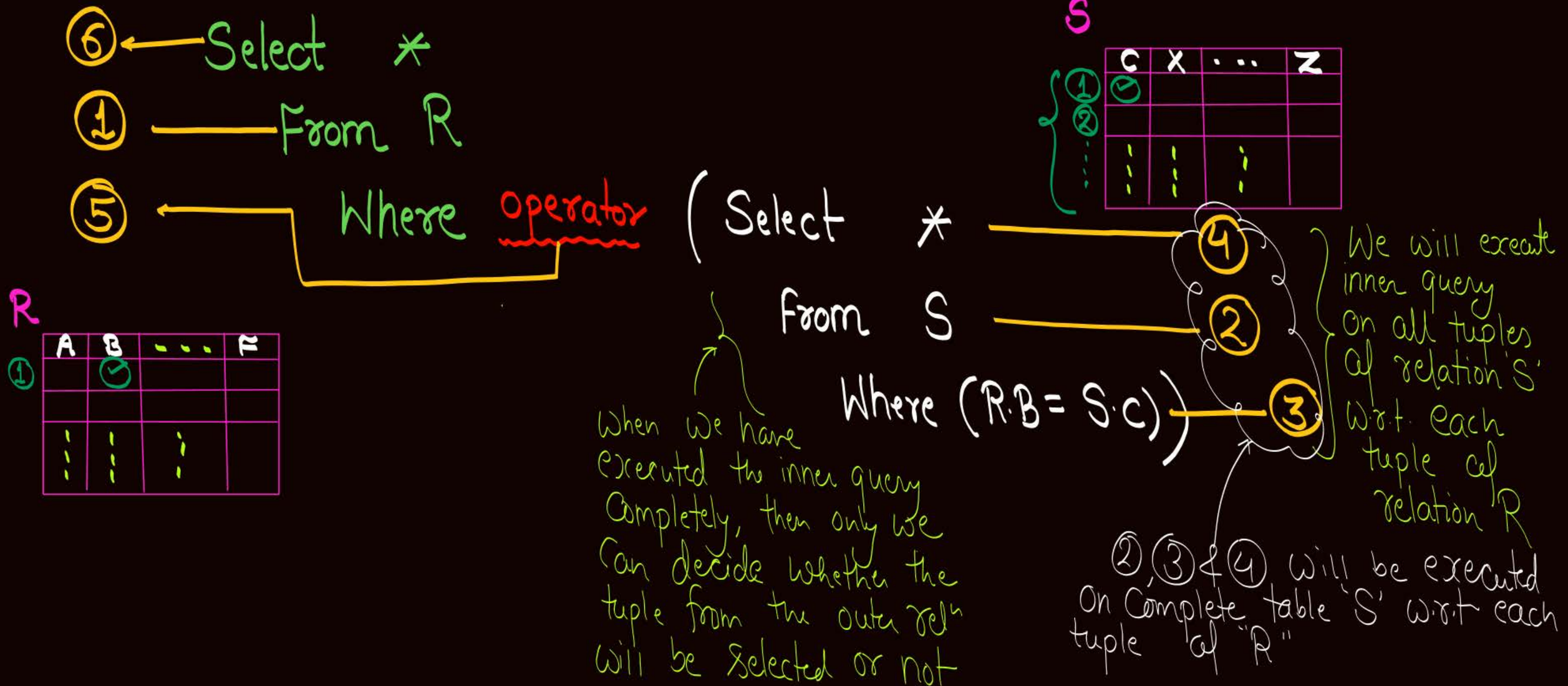
Sid	Sname	Marks	Branch
S1	A	40	CS
S2	A	20	IT
S3	B	60	CS
S4	A	60	EC
S5	C	40	IT
S6	C	NULL	EC

Correlated Sub-query :-

```
Select *
From R
Where operator ( Select *
                 From S
                 Where (R.B = S.C))
```



Order of Execution W.r.t. Correlated Sub-query :-





Topic : Operators

* IN, ANY, ALL and EXISTS are the operators that are generally used with Sub-query Concept

IN operator

Operator "IN" is used to check whether the concerned tuple is a member of a set of tuples produced by the inner query or not?

Complement of 'IN' is 'NOT IN'

Check whether

X IN { 2, 5, 8, 9, 12, 13 }

Set of tuples {i.e, o/p produced by inner query}

Select Eid From Emp

it is our Concerned tuple from outer query

(i) If X = 8, then 'IN' will return True

(ii) If X = 10, then 'IN' will return False

Check whether

X IN {(1, A), (3, D), (4, D), (7, F)}

Set of tuples (i.e, O/p produced by inner query)

Select Eid, Ename From EMP

it is our Concerned tuple from outer query

(i) If X = (4, D) then 'IN' will return True

(ii) If X = (F, 7) then 'IN' will return False

(F, 7) ≠ (7, F)

Note :-

If set of tuples is Empty {i.e, o/p produced by inner query is Empty}, then operator 'IN' will always return False

and hence operator 'NOT IN' will always return true if inner query result is Empty.

Q: Consider the following relational scheme

Supplier (Sid, Sname, Rating)

Parts (Pid, Pname, Color)

Catalog (Sid, Pid, Cost)

* Retrieve Sids of suppliers who supplied some Red Color Parts

R.A:-

$$\pi_{Catalog.Sid}\left(\sigma_{Catalog.Pid = Parts.Pid \wedge Parts.Color = 'Red'}(Catalog \times Parts)\right)$$

SQL: (without IN)

```
Select C.Sid
    From Catalog C, Parts P
        Where (C.Pid = P.Pid
                AND
               P.Color = 'Red')
```

SQL: (Using IN)

```
Select Sid From Catalog
  Where Pid IN (Select Pid From Parts
                  Where Color = 'Red')
```

'ANY' operator

Operator 'ANY' is used along with Comparison operators

$<, \leq, >, \geq, =, <>$

Check whether w.r.t. 20< x x x x x x ← No one satisfies Comparison Condn

X < ANY { 2, 5, 8, 9, 12, 14 }

w.r.t. '10<' x x x x ✓

└ we don't need to Compare with remaining elements

if X = 10, then 'ANY' will return True

if X = 20, then 'ANY' will return false

'ANY' operator

Operator 'ANY' is used along with Comparison operators

$<, \leq, >, \geq, =, <>$

Note: Operator 'ANY' will return True, if and only if at least one tuple in the set of tuples satisfy the Comparison Condition

Note: If set of tuples is empty, then operator 'ANY' will always return False

'ALL' operator

Operator 'ALL' is also used along with Comparison operators

$<, \leq, >, \geq, =, <>$

Check whether w.r.t. '1<' ✓ ✓ ✓ ✓ ✓ ✓ { No one failed the Comparison Condn.

X < ALL { 2, 5, 8, 9, 12, 14 }

w.r.t. '10<' X → we don't need to Compare with remaining elements

if X = 10, then 'ALL' will return False

if X = 1, then 'ALL' will return True

'ALL' operator

Operator 'ALL' is also used along with Comparison operators

$<, \leq, >, \geq, =, <>$

- operator 'ALL' can return false if and only if at least one tuple in the set of tuples fails the Comparison Condition
- If set of tuples is Empty, then operator 'ALL' will always return True

EXISTS operator

- EXISTS returns True if and only if inner query result is not Empty.
  - If inner query result is not Empty, then EXISTS will return True

&

If inner query result is Empty, then EXISTS will return False.

Complement of 'EXISTS' is 'NOT EXISTS'

H.W. What output will be produced by the following query if query is executed on the following instances of Catalog and Parts table

#e.g.

```
SELECT C.sid
FROM Catalog C
WHERE EXISTS ( SELECT *
               FROM Parts P
               WHERE (P.Pid=C.Pid AND P.color='RED'))
```

i.e. Correlated Sub-query

Parts

Pid	Color
P1	Red
P2	Green
P3	Red

Catalog

Sid	Pid
S1	P1
S1	P2
S2	P1
S2	P2
S3	P1
S3	P3

H.W.

#e.g.

```
SELECT C1.sid
FROM Catalog C1
WHERE NOT EXISTS ( SELECT P.Pid
                   FROM Parts P
                   WHERE NOT EXISTS (SELECT C2.Sid
                                     FROM Catalog C2
                                     WHERE(C2.Pid=P.Pid AND  C2.Sid=C1.Sid)))
```

Catalog

Sid	Pid
S_1	P_1
S_1	P_2
S_2	P_2

Parts

Pid
P_1
P_2

What output is produced by above SQL query:

A. Sids of suppliers who supplied some parts

B. Sids of suppliers who supplied only proper subset of parts from all parts

C. Sids of suppliers who supplied all parts

D. Sids of suppliers who did not supply any part



2 mins Summary

Topic SQL clauses

Topic Introduction to nested query

Topic IN, ANY, ALL and EXISTS operators

THANK - YOU